

XplagiaX — Propuesta de Expansión de Arquitectura de Plugins

Versión: v3.5+ Hoja de Ruta de Plugins | Fecha: 2026-03-21 | Autor: Pavel Santos Nunez

Mapa de Cobertura Actual

Plugin Activo	Objetivo de Detección	Brecha Conocida
<code>detector_final.py</code>	Texto AI sin modificar (41 clases de modelos via ensemble ModernBERT)	Texto AI parafraseado/ofuscado
<code>stylometric_profiler.py</code>	Huella de estilo de escritura (variabilidad, riqueza vocabulario, hapax)	Solo documento individual, sin comparación con línea base del autor
<code>hallucination_profile.py</code>	Anomalías de veracidad (vector de 25 dimensiones, cero recursos)	Sin verificación de hechos externa; solo señales estadísticas
<code>reasoning_profiler.py</code>	Trazas de cadena de pensamiento / modelos de razonamiento (o1, R1, QwQ)	Solo detecta patrones CoT explícitos; no detecta razonamiento implícito
<code>watermark_decoder.py</code>	Marcas de agua estadísticas (esquema Kirchenbauer et al.)	Solo esquemas conocidos; experimental, ruidoso

Plugins Propuestos

P0 — Prioridad Crítica (Anti-Evasión)

1. `paraphrase_detector.py` — Detección de Ofuscación y Reescritura

Problema: QuillBot, Undetectable.ai, HIX Bypass y la reescritura manual son la técnica de evasión #1 en entornos K-12 y postsecundarios. El ensemble ModernBERT actual detecta texto AI sin modificar con alta precisión pero cae a rendimiento casi aleatorio con texto parafraseado. La investigación muestra que el parafraseo estilo DIPPER reduce la precisión de DetectGPT de 70.3% a 4.6%.

Por Qué Es Necesario: Sin este plugin, XplagiaX tiene un punto ciego arquitectónico crítico que cualquier estudiante puede explotar en menos de 30 segundos. Un estudiante genera texto con ChatGPT, lo pega en QuillBot (versión gratuita), y la salida parafraseada evade completamente el ensemble ModernBERT. Esto no es una brecha teórica — es el flujo de evasión más común documentado en la literatura de integridad académica (Krishna et al., NeurIPS 2023). Cada competidor comercial que no aborda el parafraseo se vuelve irrelevante dentro de un semestre de adopción estudiantil. Para que XplagiaX sea creíble en el piloto SD5 y cualquier venta institucional, este plugin es innegociable. La inversión en el ensemble de 4 modelos se desperdicia si la salida puede ser trivialmente eludida por una extensión gratuita del navegador. Este plugin transforma XplagiaX de "detecta texto AI sin modificar" a "detecta participación de AI independientemente del post-procesamiento" — una categoría de producto fundamentalmente diferente.

Qué detecta: Firmas estadísticas que sobreviven al parafraseo — distribución de profundidad de cláusulas, anomalías de densidad de sustitución de sinónimos (las herramientas de parafraseo sobre-sustituyen), diversidad de inicio de oración, entropía de palabras de transición, estabilidad de ratio pasivo/activo, repetición de plantillas de párrafos.

Vector de características (~12 dimensiones):

- Distribución de profundidad de cláusulas (media, desviación estándar)
- Puntuación de anomalía de densidad de sinónimos
- Diversidad de bigramas de inicio de oración
- Entropía de palabras de transición
- Estabilidad de ratio pasivo-a-activo
- Puntuación de repetición de plantilla estructural de párrafo
- Gradiente de cohesión léxica (inter-oración)
- Uniformidad de ratio cobertura-a-aserción
- Huella de distribución de palabras función (4 características)

Patrón de integración:

```
PluginOrchestrator → .vectorize(text) → ParaphraseClassifier  
→ additional_analyses["paraphrase"]  
→ ForensicReportGenerator → sección HTML
```

Dependencia: Solo CPU, cero modelos externos. Opcional: embeddings DistilBERT para características de cohesión semántica.

2. `authorship_drift_profiler.py` — Comparación Longitudinal de Estilo

Problema: Para entregas repetidas (ej. caso KCA, piloto SD5), comparar un nuevo documento contra la línea base establecida del estudiante es la señal de integridad más fuerte. Un cambio repentino de estilo — salto de vocabulario, cambio de registro de formalidad, pico de complejidad de oraciones — indica autoría externa.

Por Qué Es Necesario: El análisis de documento individual tiene un techo fundamental — solo puede comparar el texto contra estadísticas genéricas de población AI/humano. No puede responder la pregunta más crítica que hace un instructor: "¿ESTE estudiante escribió ESTE texto?" La deriva de autoría es la única señal que proporciona contextualización por estudiante. En el caso KCA con Colleen Morrison, el detector existente marcó el texto como AI pero no pudo demostrar que la escritura era inconsistente con el trabajo previo del estudiante. Con un perfil de línea base construido a partir de 3-5 tareas anteriores, la detección de deriva produce evidencia que es intuitiva y defendible en reuniones de padres-profesores: "La sofisticación de vocabulario de esta entrega está 2.4 desviaciones estándar por encima del perfil de escritura establecido de su hijo/a." Esto convierte a XplagiaX de un clasificador probabilístico a un sistema de verificación de autoría — una mejora de categoría que respalda directamente la tesis de autoría basada en pulsaciones de MarkTrack Pro. Las escuelas que adoptan XplagiaX para monitoreo continuo (no detección de una sola vez) necesitan esta capacidad. También crea una ventaja competitiva: el requisito de línea base significa que los costos de cambio aumentan con cada tarea procesada.

Qué detecta: Distancia coseno entre el vector estilométrico de la entrega y el perfil establecido del estudiante. Identifica DÓNDE en el texto diverge el estilo (curva de deriva a nivel de párrafo).

Implementación: Envuelve los métodos existentes `StylometricProfiler.build_profile()` y `compare_texts()` en el pipeline. Produce una puntuación de deriva por párrafo y un índice de divergencia general.

Salida de características:

- Distancia coseno general desde la línea base
- Curva de deriva por párrafo (array de distancias)
- Puntos de quiebre de divergencia (posiciones donde el estilo cambia significativamente)
- Top 5 características que contribuyen a la divergencia
- Clasificación de riesgo: CONSISTENTE / DERIVA LEVE / DIVERGENCIA SIGNIFICATIVA

Dependencia: Requiere un `StyleProfile` pre-construido por autor. Solo CPU.

P1 — Alto Valor

3. `reference_validator.py` — Verificador de Existencia de Citas y Fuentes

Problema: La IA fabrica citas en el 18-55% de los casos dependiendo de la versión del modelo. Una cita fabricada es prueba binaria y verificable de participación de IA — la señal más accionable para oficiales de integridad académica.

Por Qué Es Necesario: Todas las demás señales de detección en XplagiaX son probabilísticas — "este texto es 87% probable generado por IA." Los veredictos probabilísticos crean espacio para disputas ("mi estilo de escritura simplemente es formal"). Una cita fabricada es binaria e irrefutable: el paper existe en CrossRef o no existe. No hay zona gris, no hay argumento, no hay apelación. Investigación de NeurIPS 2025 encontró más de 100 citas alucinadas en 53 papers aceptados que pasaron revisión por pares de 3+ revisores (Goldman, 2026). En entornos K-12 y postsecundarios, las tasas de fabricación de citas alcanzan 39-55% con ChatGPT 3.5 y 18-28% con GPT-4 (JMIR, 2024). Para el mercado objetivo de XplagiaX — oficiales de integridad académica que necesitan evidencia defendible para procedimientos formales — un validador de referencias proporciona el artefacto de prueba más fuerte posible. También aborda una brecha que ningún competidor en el espacio K-12 actualmente llena: Turnitin verifica plagio contra fuentes existentes pero no verifica si las fuentes citadas realmente existen. Este es un punto de diferenciación claro para las conversaciones de venta de XplagiaX.

Qué detecta:

- Referencias inexistentes (citas fantasma)
- DOIs incorrectos, números de volumen/número, rangos de páginas
- Referencias quiméricas (elementos de múltiples papers reales fusionados en uno)
- Discordancias autor-título
- Anacronismos de fecha (citar publicaciones futuras)

Implementación: Extracción por regex + heurísticas para formatos APA/MLA/Chicago. Validación vía API CrossRef (gratuita, 50 req/seg) y API OpenAlex (completamente abierta). Retorna: `existe` / `no_encontrado` / `ambiguo` / `quimérico` por cita.

Dependencia: Requiere acceso HTTP a CrossRef/OpenAlex. La extracción es solo CPU.

4. `register_consistency_profiler.py` — Análisis de Registro Intra-Documento

Problema: La escritura asistida por IA frecuentemente muestra cambios de formalidad no naturales: introducción casual → cuerpo hiper-formal → conclusión casual, o saltos repentinos de sofisticación de vocabulario a mitad de párrafo. Esto es distinto de `stylometric_profiler` (que perfila todo el documento) — este plugin perfila segmentos y mide la consistencia interna.

Por Qué Es Necesario: El patrón de uso indebido de IA más común en el mundo real no es "ensayo 100% generado por IA" sino "introducción y conclusión escritas por el humano con párrafos del cuerpo generados por IA." Los estudiantes escriben los primeros y últimos párrafos

ellos mismos, luego pegan la salida de ChatGPT para la parte analítica del medio. Las métricas de documento completo del pipeline actual promedian estos segmentos juntos, diluyendo la señal. El análisis de consistencia de registro detecta exactamente este patrón midiendo formalidad, sofisticación de vocabulario y complejidad sintáctica a nivel de párrafo y marcando inconsistencias internas. Un documento donde el párrafo 1 tiene un grado Flesch-Kincaid de 8 y el párrafo 3 un grado de 14 tiene un cambio de registro que es invisible para el análisis de documento completo pero obvio para el perfilado a nivel de segmento. Este plugin también detecta un segundo patrón: estudiantes que usan ChatGPT para "mejorar" su borrador — la IA pule algunos párrafos a un nivel de formalidad inconsistente con las secciones propias del estudiante. Para los instructores, la salida visual (una curva de formalidad por párrafo mostrando el salto abrupto) es inmediatamente intuitiva y no requiere alfabetización estadística para interpretar.

Vector de características (~8 dimensiones):

- Puntuación F de Heylighen por párrafo (índice de formalidad)
- Índice de sofisticación de vocabulario por segmento
- Gradiente de complejidad sintáctica
- Varianza de densidad de cobertura entre secciones
- Varianza de grado de legibilidad (Flesch-Kincaid por párrafo)
- Puntos de quiebre de cambio de registro
- Coeficiente de variación de formalidad
- Varianza de profundidad de subordinación de cláusulas

Dependencia: Solo CPU. Opcional: spaCy para profundidad sintáctica.

5. `prompt_signature_profiler.py` — Reconstrucción de Patrones de Prompt

Problema: Ciertos patrones de prompt dejan trazas detectables. "Escribe un ensayo sobre X en 500 palabras" produce texto que refleja la estructura del prompt — exactamente 5 párrafos, densidad de oraciones temáticas, conclusión que refleja la introducción.

Por Qué Es Necesario: Incluso cuando las herramientas de parafraseo logran modificar la superficie léxica del texto AI, la huella estructural del prompt original sobrevive. Un ensayo de 5 párrafos con párrafos de 100 palabras perfectamente equilibrados, cada uno comenzando con una oración temática, y una conclusión que refleja la introducción no es como los humanos escriben naturalmente — es como ChatGPT responde a "Escribe un ensayo de 500 palabras sobre X." Esta firma estructural es ortogonal a todas las demás señales de detección en el pipeline: el ensemble ModernBERT detecta patrones AI a nivel de token, el profiler estilométrico mide características de vocabulario, el profiler de razonamiento detecta marcadores CoT, pero

ninguno de ellos detecta conformidad estructural a nivel de prompt. Este plugin llena una dimensión analítica única. También es la señal más difícil de derrotar para los estudiantes — incluso si parafrasean cada oración y cambian el vocabulario, la plantilla de 5 párrafos, la precisión de enumeración y los marcadores de meta-discurso ("En este ensayo exploraremos...") persisten porque cambiar la estructura del documento requiere un esfuerzo compositivo genuino que la mayoría de las herramientas de evasión no realizan. Para K-12 específicamente, las firmas de prompt son altamente prevalentes porque los estudiantes usan prompts simples que inducen plantillas.

Vector de características (~10 dimensiones):

- Puntuación de conformidad con plantilla (ensayo de 5 párrafos, lista-de-N, pros-y-contras)
- Precisión de enumeración (¿el texto enumera exactamente N elementos cuando el tema no se descompone naturalmente así?)
- Densidad de oración temática por párrafo
- Ratio conclusión-refleja-introducción
- Discordancia conteo de párrafos vs. complejidad del tema
- Detección de eco instruccional (lenguaje residual del prompt)
- Conformidad de conteo de palabras a números redondos
- Uniformidad de balance de secciones (todos los párrafos ~misma longitud)
- Ratio de meta-discurso ("En este ensayo", "En conclusión")
- Puntuación de ausencia de saludo/despedida

Dependencia: Solo CPU.

P2 — Adiciones Estratégicas

6. `multilingual_transfer_detector.py` — Detección de Evasión Cross-Lingüística

Problema: Técnica de evasión: generar en inglés → traducir al idioma objetivo. Los artefactos de traducción son detectables: colocaciones no naturales, patrones de calco, mal uso de preposiciones, anomalías de colocación de artículos. Crítico para la red multilingüe de LCI Education.

Por Qué Es Necesario: LCI Education opera en múltiples países e idiomas. El pipeline actual es anglo-céntrico — el ensemble ModernBERT fue entrenado en corpus en inglés, y los profilers estilométrico/razonamiento usan listas de stopwords en inglés y patrones regex en inglés. Los estudiantes en programas de francés, español o portugués pueden generar texto en inglés (donde los modelos AI son más fuertes), pasarlo por Google Translate o DeepL, y entregar la traducción. El pipeline actual tiene cero capacidad de detección contra este flujo de trabajo. Los

artefactos de traducción son lingüísticamente distintos de los artefactos de parafraseo: patrones de calco (traducciones literales de expresiones inglesas que no existen en el idioma objetivo), mal uso de preposiciones (lógica preposicional del inglés aplicada a lenguas romances) y colocación no natural de artículos son todos detectables con extractores de características específicos por idioma. Para que XplagiaX sea viable como producto empresarial para la red global de LCI — no solo los campus de habla inglesa — este plugin es requerido. También abre los mercados educativos latinoamericanos y europeos donde las herramientas de detección de AI son aún más escasas que en Norteamérica.

Vector de características: Puntuación de naturalidad de colocaciones, anomalía de distribución de preposiciones, patrones de uso de artículos, densidad de calco, frecuencia de falsos cognados.

Dependencia: spaCy con modelos multilingües.

7. `metadata_forensics.py` — Análisis de Metadatos de Documento

Problema: Analiza metadatos del contenedor .docx/.pdf en lugar del contenido textual. Detecta: brecha creación-a-última-modificación (indicador de copiar-pegar), firma de aplicación, ratio tiempo de edición vs. conteo de palabras, conteo de revisiones.

Por Qué Es Necesario: Todos los demás plugins en XplagiaX analizan contenido textual. La forensia de metadatos opera en un canal de evidencia completamente independiente — el contenedor del documento mismo. Un archivo .docx cuya marca de tiempo de creación es 11:42 PM, marca de tiempo de última modificación es 11:43 PM, tiene 1 revisión y contiene 2,000 palabras no fue escrito por un humano — fue pegado. Un documento creado por "Google Docs" pero entregado como .docx tiene una huella de metadatos diferente a uno creado en Microsoft Word. Un PDF generado directamente por un script de Python (común en pipelines de AI) lleva metadatos de aplicación como "ReportLab" o "FPDF" en lugar de "Microsoft Word" o "LibreOffice." Estas señales son invisibles para el análisis de texto e imposibles de falsificar sin herramientas especializadas que los estudiantes típicamente no poseen. La evidencia de metadatos también es altamente convincente en procedimientos formales porque es objetiva y no probabilística: la marca de tiempo muestra una ventana de edición de 1 minuto o no. Combinada con el seguimiento de pulsaciones de MarkTrack Pro, la forensia de metadatos crea una cadena de evidencia conductual multi-capa que va mucho más allá del análisis de contenido solo.

Dependencia: python-docx, PyPDF2. Sin modelos ML.

8. `perplexity_profiler.py` — Análisis de Perplejidad con LM de Referencia

Problema: Curva de perplejidad por token usando GPT-2 small como LM de referencia. Actualmente el pipeline aproxima la perplejidad via características estadísticas. Una curva de

perplejidad adecuada con análisis de variabilidad es una señal más fuerte.

Por Qué Es Necesario: La perplejidad — cuán "sorprendido" está un modelo de lenguaje por cada token — es la señal fundamental que subyace a DetectGPT, Fast-DetectGPT, Binoculars y la mayoría de métodos de detección zero-shot. El pipeline actual de XplagiaX aproxima la perplejidad indirectamente a través de proxies estadísticos (entropía, variabilidad), pero estos son de granularidad gruesa y pierden la granularidad por token que hace las curvas de perplejidad tan poderosas. Un profiler de perplejidad apropiado produce una curva mostrando exactamente DÓNDE en el texto el modelo de lenguaje no se sorprende (baja perplejidad = probablemente generado por AI) versus sorprendido (alta perplejidad = probablemente escrito por humano). Esta resolución por token habilita detección híbrida: incluso en un documento donde 70% es escrito por humano, la sección 30% generada por AI mostrará un valle de perplejidad distinto que las métricas a nivel de párrafo no detectan. La curva de perplejidad también es una de las señales más interpretables para educadores — "esta sección es altamente predecible para un modelo de lenguaje, lo que significa que sigue los patrones estadísticos que los modelos AI producen" es una explicación clara. Agregar este plugin también hace que la pila de detección de XplagiaX sea comparable a los sistemas de grado investigativo (DetectGPT, Binoculars) usados en papers de detección de AI revisados por pares, fortaleciendo la credibilidad académica del producto.

Dependencia: GPT-2 small (~500MB). GPU recomendada.

Hoja de Ruta de Implementación

Fase	Plugins	Cronología	ROI
Fase 1	paraphrase_detector, authorship_drift_profiler	Inmediata	Más alto — cierra la brecha de evasión #1
Fase 2	reference_validator, register_consistency_profiler	Próximo sprint	Alto — prueba binaria para citas
Fase 3	prompt_signature_profiler, metadata_forensics, multilingual_transfer_detector	Q2 2026	Medio — diferenciación
Fase 4	perplexity_profiler	Cuando el presupuesto de GPU lo permita	Medio — detección mejorada

Extensión de PluginConfig

python

```
@dataclass
```

```
class PluginConfig:
```

```
    # ... flags existentes ...
```

```
    enable_paraphrase: bool = True      # P0 – anti-evasión
```

```
    enable_authorship_drift: bool = False # P0 – requiere línea base
```

```
    enable_reference_check: bool = False # P1 – requiere red
```

```
    enable_register: bool = True        # P1 – análisis de formalidad
```

```
    enable_prompt_signature: bool = True # P1 – traza de prompt
```

```
    enable_metadata: bool = False       # P2 – requiere ruta de archivo
```

```
    enable_multilingual: bool = False   # P2 – requiere spaCy multi
```

```
    enable_perplexity: bool = False     # P2 – requiere GPT-2
```

```
    baseline_profile: Optional[Any] = None # para comparación de deriva
```